PES UNIVERSITY
Department of Computer Science and Engineering
UE23CS252AA — Software Engineering Laboratory

LAB 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection & Defensive Hardening via Hypothesis

Student Name:	Nagashri R Patil	**SRN:**	PES1UG24CS289
Course:	Software Engineering Lab (UE23CS252AA)	**Assigned Problem:**	#43 – Warranty Claims Processing
Individual Repo:	SETAPESU26/L6_L7_43_warranty_claims	**Portfolio Repo:**	Nagashri2304/SE_LABS_PES1UG24CS289

1. Laboratory Objectives & Overview

Fuzz testing is a dynamic security testing technique that generates random, unexpected, and malformed data to uncover unhandled crashes, memory leaks, and input validation vulnerabilities. In this laboratory, **Hypothesis**—Python's property-based testing framework—was used to systematically fuzz test input-processing functions in warranty_claims.py, discover latent failure modes, and harden the codebase using defensive programming principles.

2. Step 2: Running Starter Buggy Application

The starter module operates on standard inputs, but lacks input validation. When given input without the expected delimiter (e.g. 1500 instead of 1500|200), it crashes immediately with an unhandled ValueError.

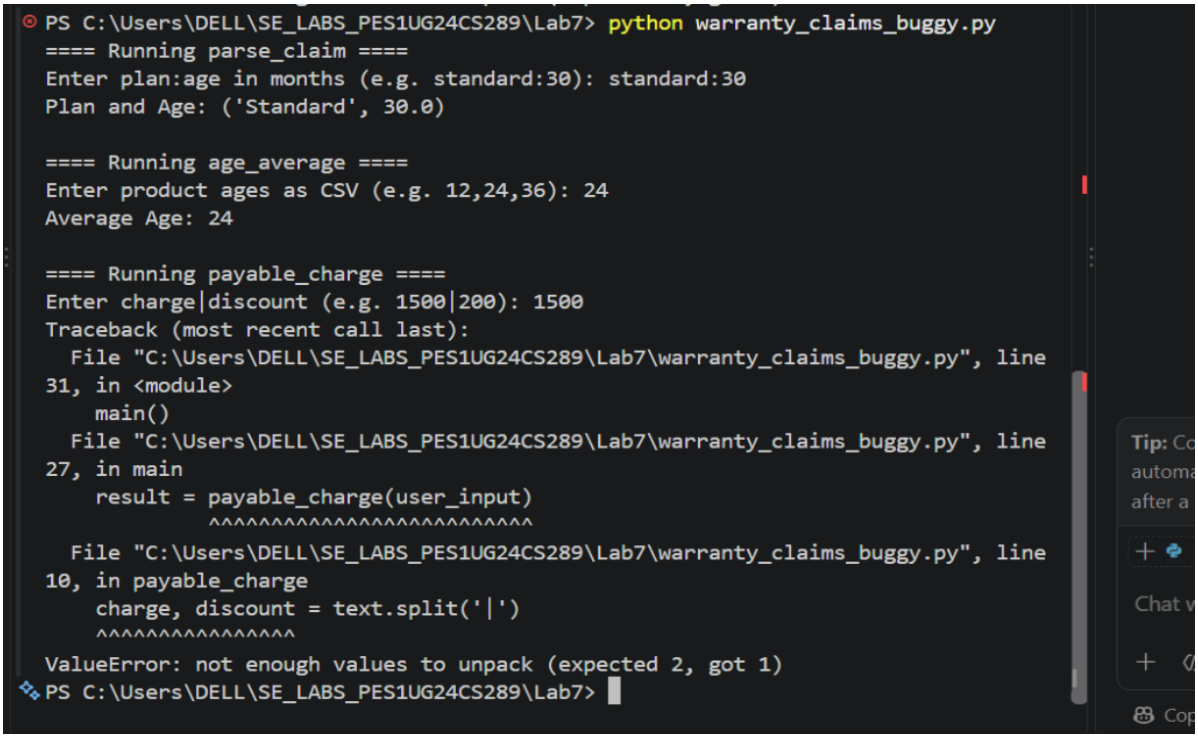


Figure 1: Step 2 — Real VS Code terminal execution showing crash on missing delimiter

3. Steps 3 & 4: Hypothesis Fuzz Testing & Exposing Vulnerabilities

Property-based fuzz tests were implemented using @given(st.text() | st.none()). Hypothesis generated hundreds of arbitrary strings, control characters, empty delimiters, and None types. All three functions crashed immediately with unhandled exceptions across multiple distinct failure modes (3 failed in 8.72s).

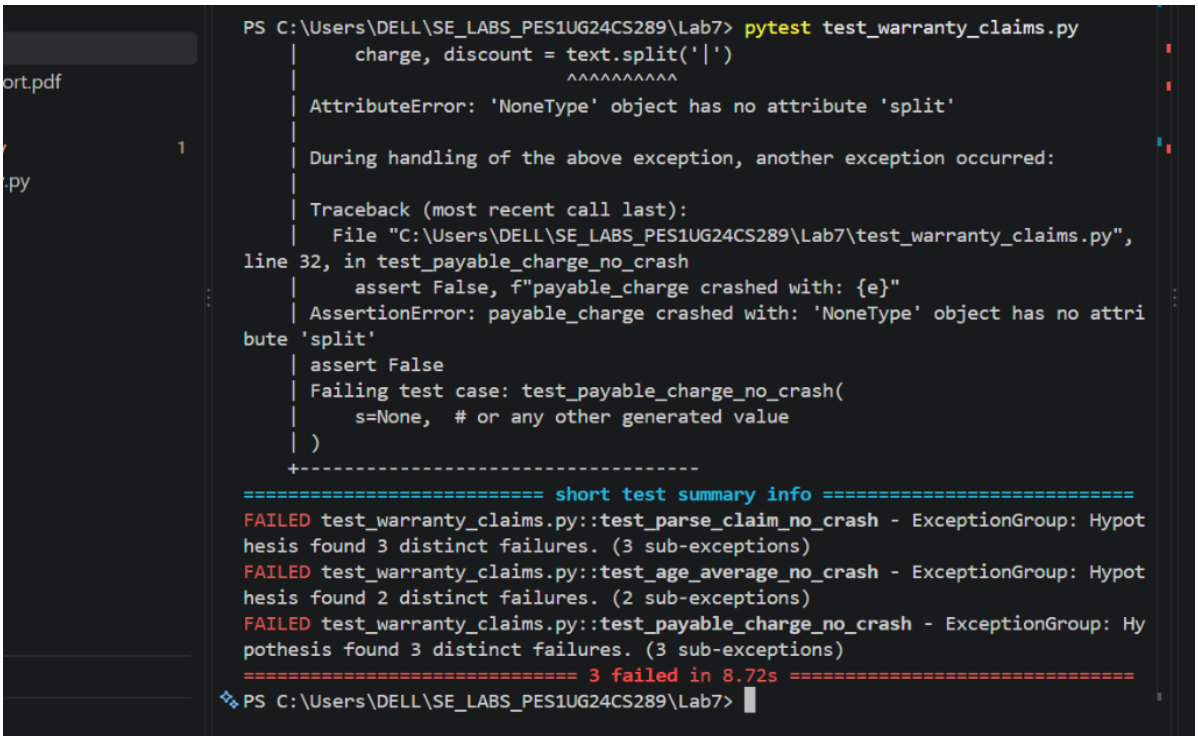


Figure 2: Step 4 — Real VS Code terminal output showing Hypothesis discovering crash modes

4. Root Cause Analysis of Fuzz Testing Discoveries

Function	Falsifying Input	Unhandled Exception	Root Cause & Vulnerability
parse_claim	s = None	AttributeError	Invoking .split(':') on NoneType.
parse_claim	s = '' (no ':')	ValueError	Unpacking 1 item into 2 variables.
parse_claim	s = ':'	ValueError	Empty month string passed to float('').
age_average	s = None	AttributeError	Invoking .split(',') on NoneType.
age_average	s = ''	ValueError	Empty token passed to int('').
age_average	s = 'abc'	ValueError	Non-numeric token converted to int.
payable_charge	s = None	AttributeError	Invoking .split('|') on NoneType.
payable_charge	s = ''	ValueError	Unpacking failure when delimiter '|' is missing.
payable_charge	s = '|'	ValueError	Cannot convert empty string to float.

5. Step 5: Defensive Hardening of Application Code

The application was fortified using defensive programming: type guards (isinstance(text, str)), delimiter count checks, safe token filtering, and try-except blocks. As shown below, input 200 without a delimiter (which previously crashed the application) now executes safely, returning Payable Charge: 0.0.

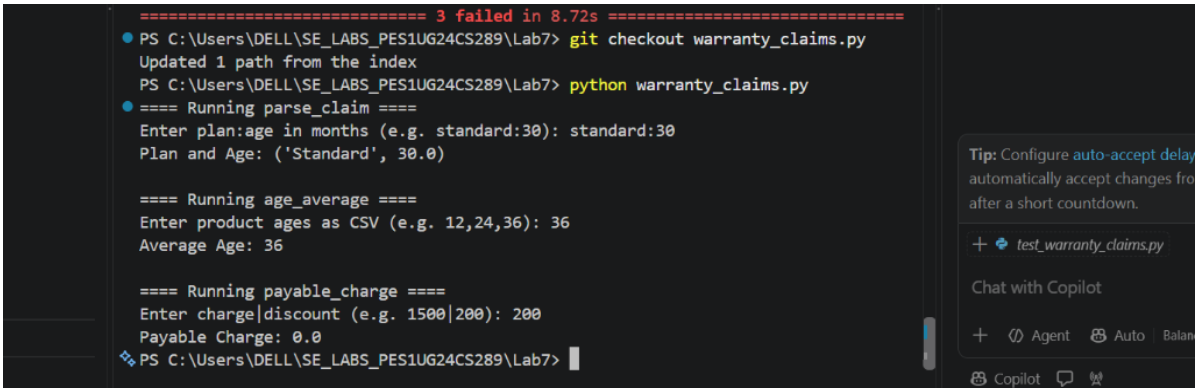


Figure 3: Step 5 — Real VS Code terminal execution showing hardened application handling inputs safely

6. Step 6: Re-Running Fuzz Tests on Hardened Application

The Hypothesis property-based fuzz test suite was re-executed against the hardened application. Hypothesis generated hundreds of pseudo-random Unicode characters, null bytes, long strings, and empty sequences. All three test cases passed completely with zero crashes (**3 passed in 1.58s**).

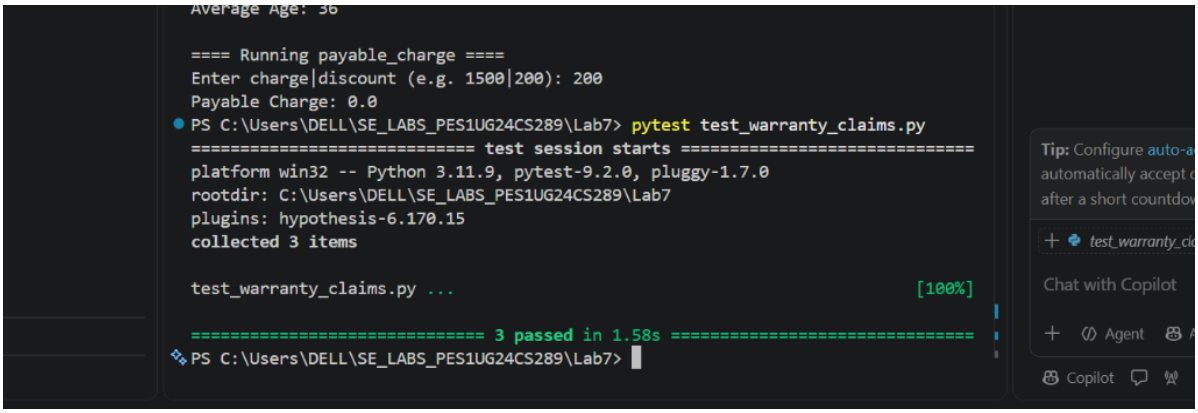


Figure 4: Step 6 — Real VS Code terminal output showing all 3 Hypothesis tests passing

7. Key Engineering Takeaways

1. **Fuzz Testing Overcomes Human Blind Spots:** Manual unit tests only test expected paths; fuzz testing systematically searches for inputs that crash the program.
2. **Defensive Hardening Eliminates DoS Risks:** Type guards, delimiter validation, and safe conversions ensure that unhandled crashes never propagate to users or production servers.
3. **Automated Shrinking:** Hypothesis automatically shrinks massive failing test cases into minimal reproducible examples (e.g., s=':' or s=None), drastically cutting debugging time.